

Data Analysis Algorithms

Week-5

Task

Dhusyanth R

CH.SC.U4CSE24161

1.) Quick Sort:

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  void swap(int* a, int* b) {
4      int t = *a;
5      *a = *b;
6      *b = t;
7  }
8  int partition(int arr[], int low, int high, int pivotChoice) {
9      int pivotIndex;
10     if (pivotChoice == 1) {
11         pivotIndex = low;
12     } else if (pivotChoice == 2) {
13         pivotIndex = high;
14     } else {
15         pivotIndex = low + rand() % (high - low + 1);
16         printf("Random Pivot Element Chosen: %d\n", arr[pivotIndex]);
17     }
18     swap(&arr[pivotIndex], &arr[high]);
19     int pivot = arr[high];
20     int i = (low - 1);
21     for (int j = low; j <= high - 1; j++) {
22         if (arr[j] < pivot) {
23             i++;
24             swap(&arr[i], &arr[j]);
25         }
26     }
27     swap(&arr[i + 1], &arr[high]);
28     return (i + 1);
29 }
30 void quickSort(int arr[], int low, int high, int pivotChoice) {
31     if (low < high) {
32         int pi = partition(arr, low, high, pivotChoice);
33         quickSort(arr, low, pi - 1, pivotChoice);
34         quickSort(arr, pi + 1, high, pivotChoice);
35     }
36 }
```

```

36 }
37 int main() {
38     printf("CH.SC.U4CSE24161\n");
39     printf("Enter number of elements: ");
40     if (scanf("%d", &n) != 1) return 1;
41     int *arr = (int *)malloc(n * sizeof(int));
42     printf("Enter %d elements: ", n);
43     for (int i = 0; i < n; i++) {
44         scanf("%d", &arr[i]);
45     }
46     printf("Choose Pivot Element:\n1. First Element\n2. Last Element\n3. Random Element\nChoice: ");
47     scanf("%d", &pivotChoice);
48     quickSort(arr, 0, n - 1, pivotChoice);
49     printf("Sorted array: ");
50     for (int i = 0; i < n; i++) {
51         printf("%d ", arr[i]);
52     }
53     printf("\n");
54     free(arr);
55     return 0;
56 }
57 }

```

Output:

```

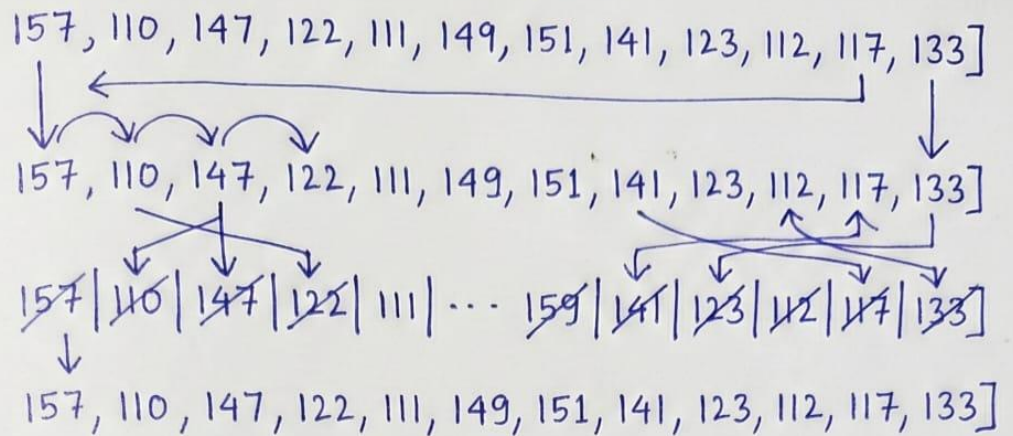
CH.SC.U4CSE24161
Enter number of elements: 3
Enter 3 elements: 1
2
3
Choose Pivot Element:
1. First Element
2. Last Element
3. Random Element
Choice: 2
Sorted array: 1 2 3

-----
Process exited after 8.439 seconds with return value 0
Press any key to continue . . . |

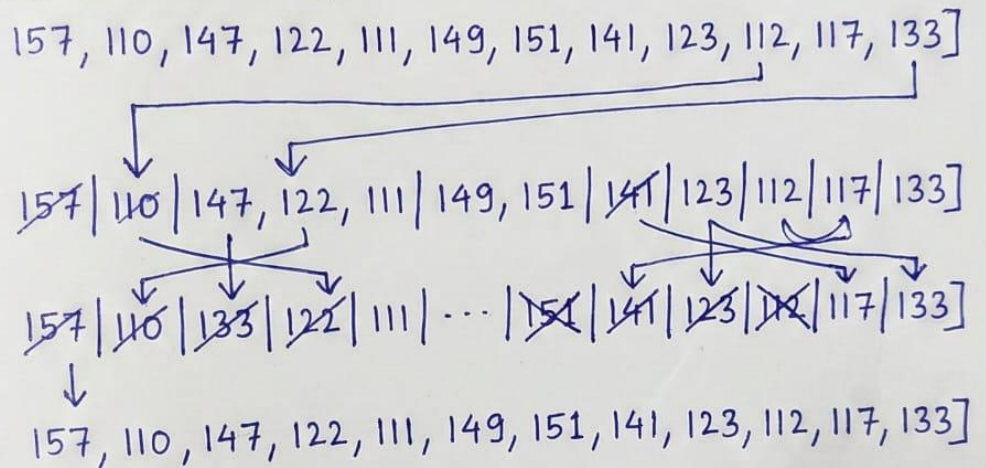
```

The handwritten algorithm for this code:

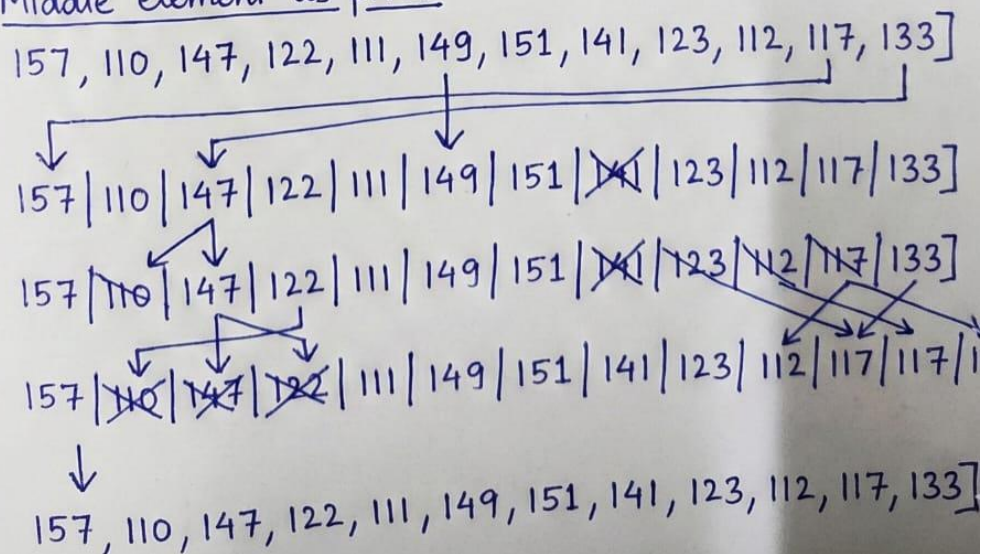
Quick Sort: First element as pivot



Last element as pivot



Middle element as pivot



Time Complexity: $O(n \log n)$ (Average) / $O(n^2)$

(Worst Case) **Justification:** The algorithm uses a divide-and-conquer strategy. In the average case (especially with the Random Pivot choice), the array is split into two nearly equal halves, requiring $\log n$ levels of recursion, with each level performing a linear $O(n)$ partition. However, if the pivot choice consistently results in highly unbalanced splits (e.g., choosing the first or last element of an already sorted array), the recursion depth can reach n , leading to a worst-case performance of $O(n^2)$

Space Complexity: $O(\log n)$ (Average) / $O(n)$ (Worst Case)

Justification: The space complexity is determined by the maximum depth of the function call stack during recursion. Since an AVL tree is strictly balanced, its height h is guaranteed to be $O(\log n)$. Even though you start with an unbalanced tree, the recursive depth of `balanceTree` or `printLevelOrder` will not exceed the initial height of the tree. No significant auxiliary data structures (like arrays or queues) are used, only the memory for the nodes themselves and the recursion stack.